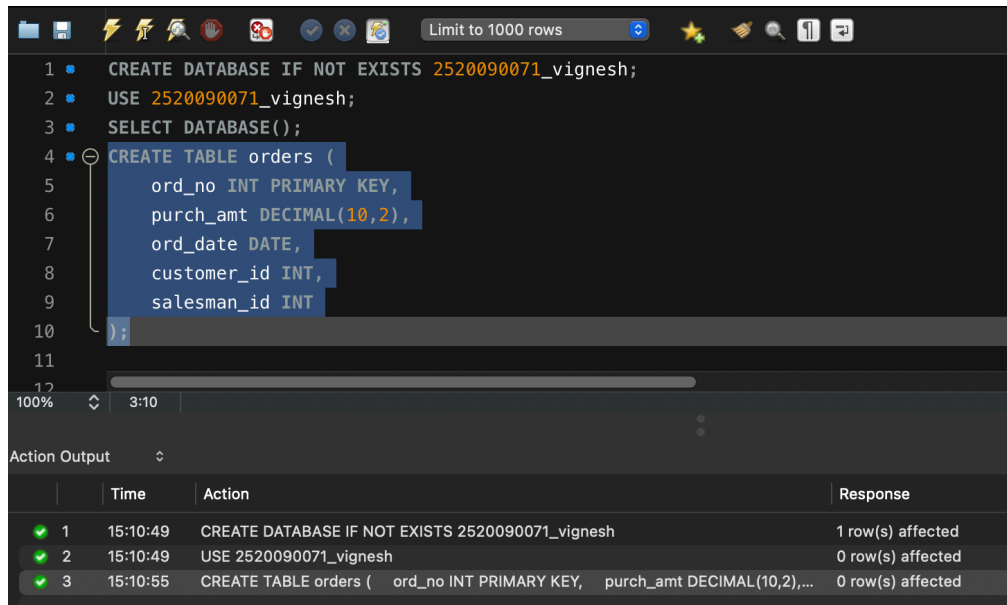


SKILL WEEK-2

Step1: Create Table



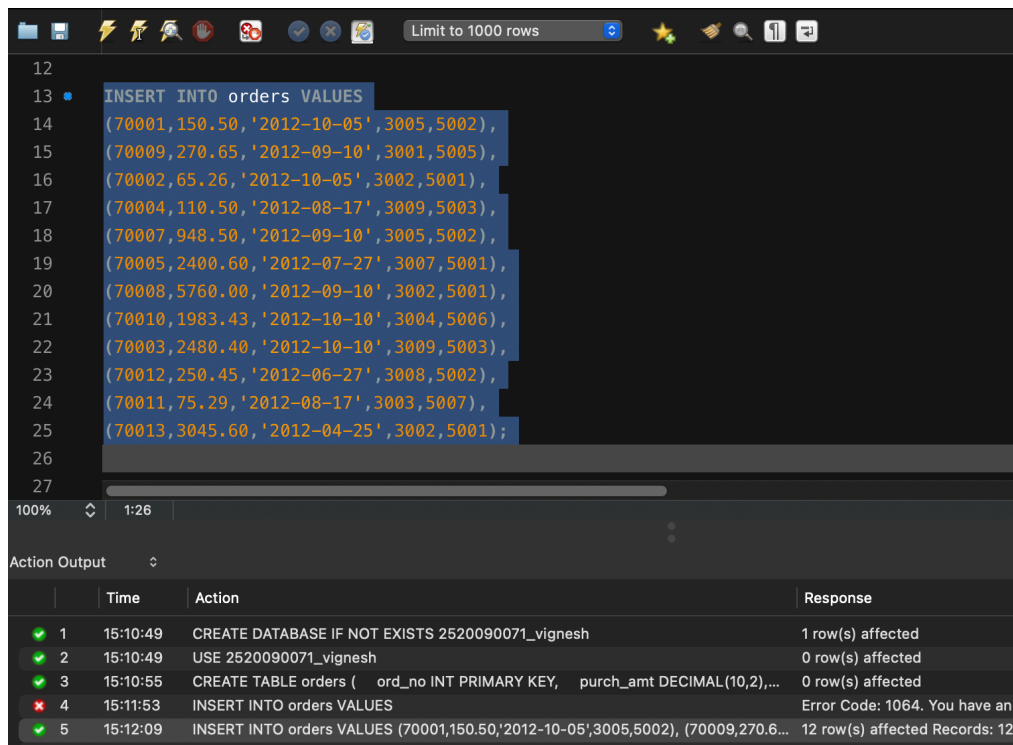
The screenshot shows a SQL IDE with a dark theme. The top toolbar includes icons for file operations, execution, and search. A dropdown menu is set to "Limit to 1000 rows". The SQL editor contains the following code:

```
1 CREATE DATABASE IF NOT EXISTS 2520090071_vignesh;  
2 USE 2520090071_vignesh;  
3 SELECT DATABASE();  
4 CREATE TABLE orders (  
5     ord_no INT PRIMARY KEY,  
6     purch_amt DECIMAL(10,2),  
7     ord_date DATE,  
8     customer_id INT,  
9     salesman_id INT  
10 );
```

Below the editor is the "Action Output" panel, which displays a table of execution results:

	Time	Action	Response
✓ 1	15:10:49	CREATE DATABASE IF NOT EXISTS 2520090071_vignesh	1 row(s) affected
✓ 2	15:10:49	USE 2520090071_vignesh	0 row(s) affected
✓ 3	15:10:55	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2),...	0 row(s) affected

Step2: Insert Data



The screenshot shows the same SQL IDE with the following code in the editor:

```
12  
13 INSERT INTO orders VALUES  
14 (70001,150.50,'2012-10-05',3005,5002),  
15 (70009,270.65,'2012-09-10',3001,5005),  
16 (70002,65.26,'2012-10-05',3002,5001),  
17 (70004,110.50,'2012-08-17',3009,5003),  
18 (70007,948.50,'2012-09-10',3005,5002),  
19 (70005,2400.60,'2012-07-27',3007,5001),  
20 (70008,5760.00,'2012-09-10',3002,5001),  
21 (70010,1983.43,'2012-10-10',3004,5006),  
22 (70003,2480.40,'2012-10-10',3009,5003),  
23 (70012,250.45,'2012-06-27',3008,5002),  
24 (70011,75.29,'2012-08-17',3003,5007),  
25 (70013,3045.60,'2012-04-25',3002,5001);  
26  
27
```

The "Action Output" panel shows the following results:

	Time	Action	Response
✓ 1	15:10:49	CREATE DATABASE IF NOT EXISTS 2520090071_vignesh	1 row(s) affected
✓ 2	15:10:49	USE 2520090071_vignesh	0 row(s) affected
✓ 3	15:10:55	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2),...	0 row(s) affected
✗ 4	15:11:53	INSERT INTO orders VALUES	Error Code: 1064. You have an
✓ 5	15:12:09	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.6...	12 row(s) affected Records: 12

Query1: WHERE CLAUSE

Finance wants all orders whose purchase amount is greater than \$2,000

The screenshot shows a database IDE with a SQL editor and a results grid. The SQL editor contains the following code:

```
24 (70011,75.29,'2012-08-17',3003,5007),
25 (70013,3045.60,'2012-04-25',3002,5001);
26
27
28 • SELECT * FROM orders
29 WHERE purch_amt > 2000;
```

The results grid displays the following data:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
HULL	HULL	HULL	HULL	HULL

The Action Output section shows the following log:

	Time	Action	Response
✓ 1	15:10:49	CREATE DATABASE IF NOT EXISTS 2520090071_vignesh	1 row(s) affected
✓ 2	15:10:49	USE 2520090071_vignesh	0 row(s) affected
✓ 3	15:10:55	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2),...	0 row(s) affected
✗ 4	15:11:53	INSERT INTO orders VALUES	Error Code: 1064. You have an
✓ 5	15:12:09	INSERT INTO orders VALUES (70001,150.50,'2012-10-05';3005,5002), (70009,270.6...	12 row(s) affected Records: 12
✓ 6	15:13:08	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned

Query2: WHERE CLAUSE

Management wants all orders placed on 2012-09-10.

The screenshot shows a database IDE with a SQL editor and a results grid. The SQL editor contains the following code:

```
29 WHERE purch_amt > 2000;
30
31 • SELECT * FROM orders
32 WHERE ord_date='2012-09-10';
33
```

The results grid displays the following data:

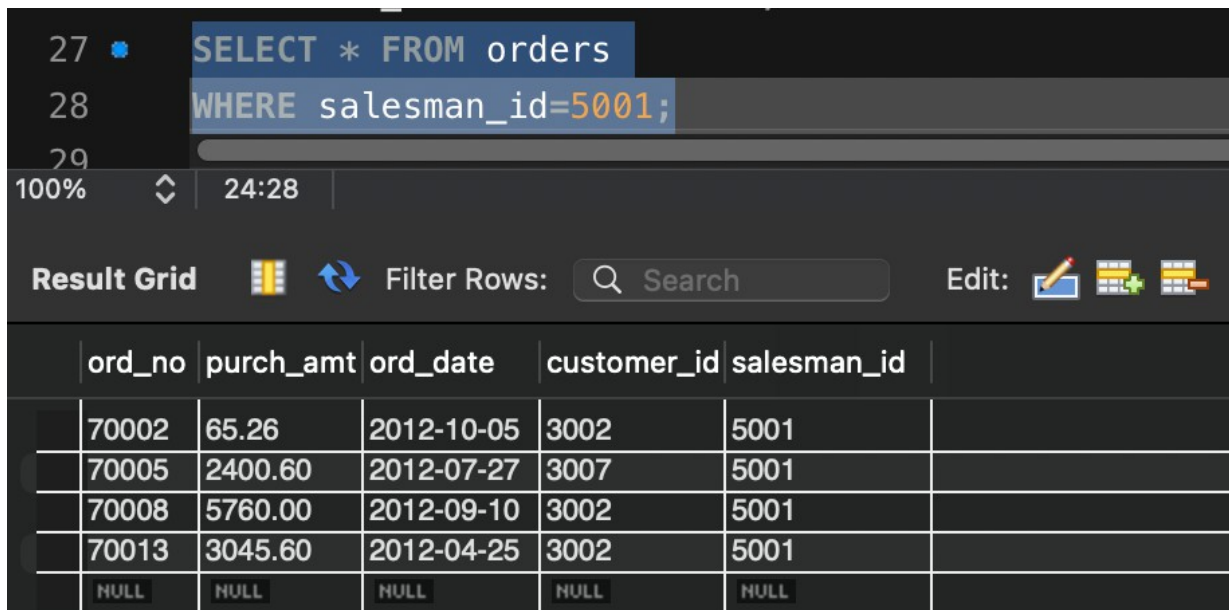
ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
HULL	HULL	HULL	HULL	HULL

The Action Output section shows the following log:

	Time	Action	Response
✓ 1	15:10:49	CREATE DATABASE IF NOT EXISTS 2520090071_vignesh	1 row(s) affected
✓ 2	15:10:49	USE 2520090071_vignesh	0 row(s) affected
✓ 3	15:10:55	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2),...	0 row(s) affected
✗ 4	15:11:53	INSERT INTO orders VALUES	Error Code: 1064. You have an
✓ 5	15:12:09	INSERT INTO orders VALUES (70001,150.50,'2012-10-05';3005,5002), (70009,270.6...	12 row(s) affected Records: 12
✓ 6	15:13:08	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
✓ 7	15:14:26	SELECT * FROM orders WHERE ord_date='2012-09-10' LIMIT 0, 1000	3 row(s) returned

Query 3 : WHERE CLAUSE

Find all orders handled by salesman 5001

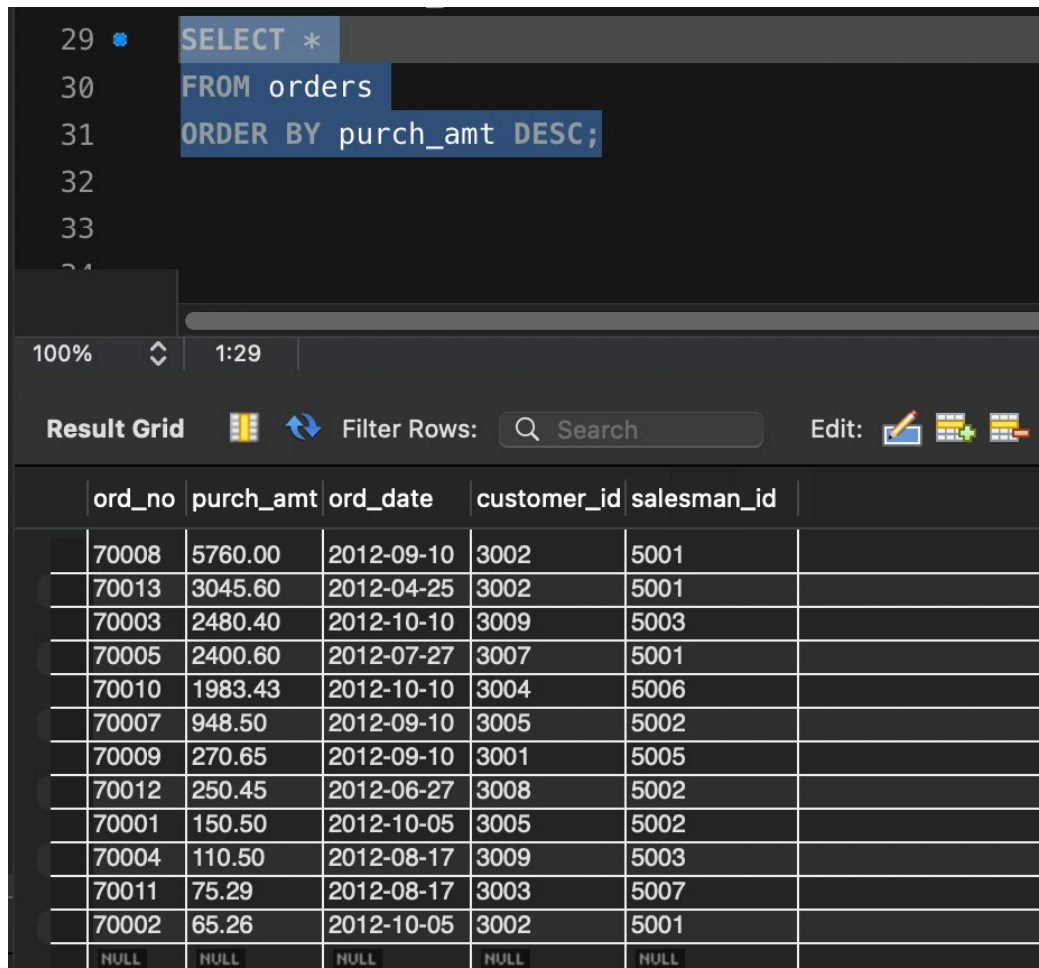


The screenshot shows a SQL IDE with a query editor and a result grid. The query is: `SELECT * FROM orders WHERE salesman_id=5001;`. The result grid displays 5 rows of data, including columns for order number, purchase amount, order date, customer ID, and salesman ID.

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	65.26	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
NULL	NULL	NULL	NULL	NULL

Query 4 : ORDER BY

CEO wants to see orders ranked from highest amount to lowest amount



The screenshot shows a SQL IDE with a query and its results. The query is: `SELECT * FROM orders ORDER BY purch_amt DESC;`. The result grid displays 14 rows of data, sorted by purchase amount in descending order. The columns are the same as in Query 3.

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70002	65.26	2012-10-05	3002	5001
NULL	NULL	NULL	NULL	NULL

Query 5 : ORDER BY

Operations team wants oldest orders first.

```
32 SELECT *
33 FROM orders
34 ORDER BY ord_date;
```

100% 19:34

Result Grid Filter Rows: Search Edit:

	ord_no	purch_amt	ord_date	customer_id	salesman_id	
	70013	3045.60	2012-04-25	3002	5001	
	70012	250.45	2012-06-27	3008	5002	
	70005	2400.60	2012-07-27	3007	5001	
	70004	110.50	2012-08-17	3009	5003	
	70011	75.29	2012-08-17	3003	5007	
	70007	948.50	2012-09-10	3005	5002	
	70008	5760.00	2012-09-10	3002	5001	
	70009	270.65	2012-09-10	3001	5005	
	70001	150.50	2012-10-05	3005	5002	
	70002	65.26	2012-10-05	3002	5001	
	70003	2480.40	2012-10-10	3009	5003	
	70010	1983.43	2012-10-10	3004	5006	
	NULL	NULL	NULL	NULL	NULL	

Query 6 : SUM()

Finance department wants total revenue generated.

```
35 SELECT SUM(purch_amt) AS total_revenue
36 FROM orders;
```

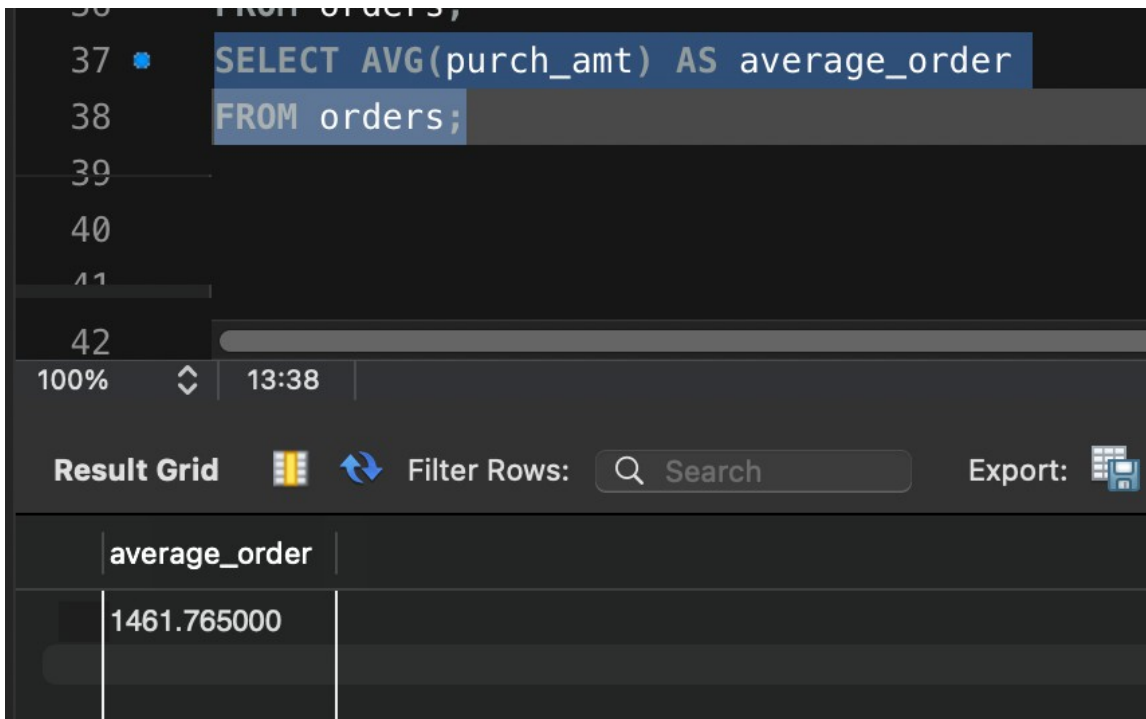
100% 13:36

Result Grid Filter Rows: Search Export:

	total_revenue	
	17541.18	

Query 7 : AVG()

Management wants average order value



```
37 SELECT AVG(purch_amt) AS average_order
38 FROM orders;
```

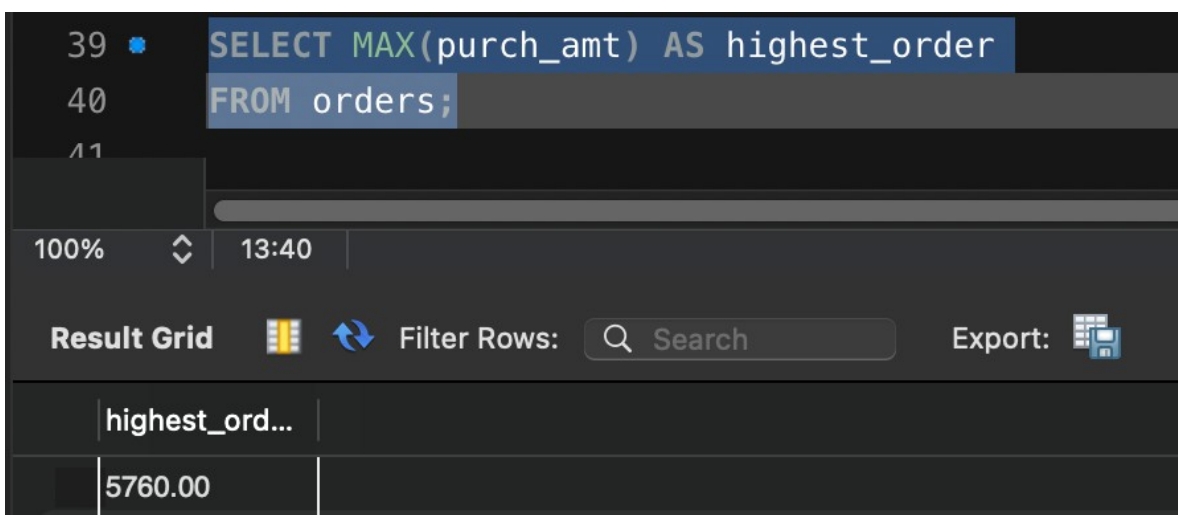
100% 13:38

Result Grid Filter Rows: Search Export:

average_order
1461.765000

Query 8 : MAX()

Find the highest order amount



```
39 SELECT MAX(purch_amt) AS highest_order
40 FROM orders;
```

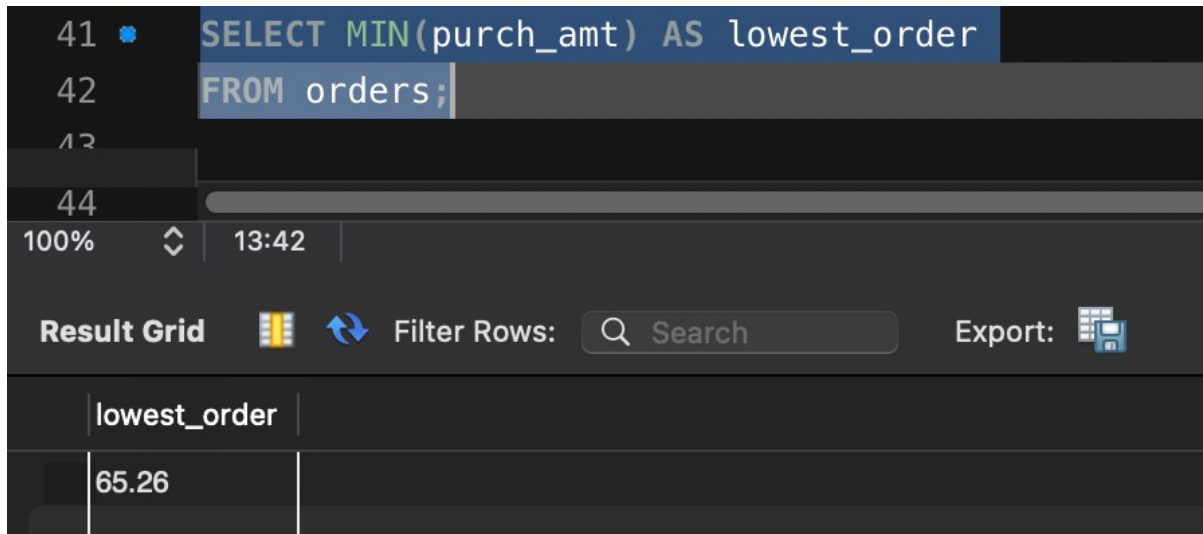
100% 13:40

Result Grid Filter Rows: Search Export:

highest_ord...
5760.00

Query 9 : MIN()

Find the lowest order amount



The screenshot shows a SQL query editor with the following SQL code:

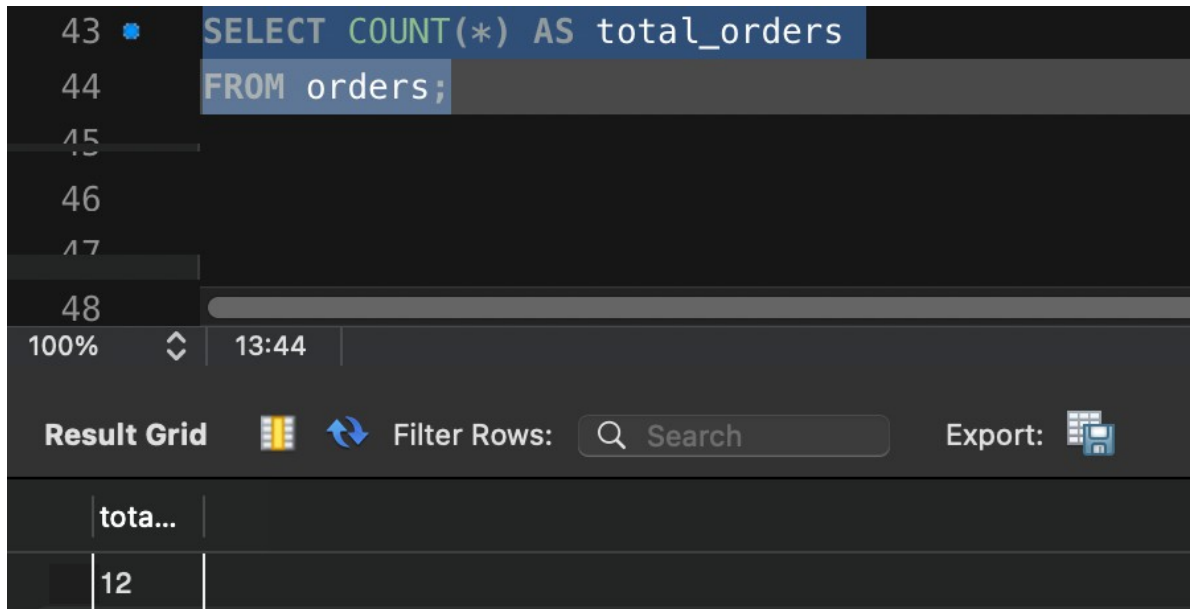
```
41 SELECT MIN(purch_amt) AS lowest_order
42 FROM orders;
```

Below the query editor, the 'Result Grid' is displayed. It shows a single column named 'lowest_order' with a single row containing the value '65.26'.

lowest_order
65.26

Query 10 : COUNT()

Find total number of orders



The screenshot shows a SQL query editor with the following SQL code:

```
43 SELECT COUNT(*) AS total_orders
44 FROM orders;
```

Below the query editor, the 'Result Grid' is displayed. It shows a single column named 'total...' (truncated from 'total_orders') with a single row containing the value '12'.

total...
12

Query 11 : GROUP BY

Sales director wants total sales generated by each salesman

```
45 SELECT salesman_id,
46         SUM(purch_amt) AS total_sales
47 FROM orders
48 GROUP BY salesman_id;
```

100% 22:48

Result Grid Filter Rows: Search Export:

salesman_id	total_sales
5002	1349.45
5001	11271.46
5003	2590.90
5005	270.65
5006	1983.43
5007	75.29

Query 12 : GROUP BY

Marketing wants total purchases made by each customer

```
49 SELECT customer_id,
50         SUM(purch_amt) AS total_purchase
51 FROM orders
52 GROUP BY customer_id;
```

100% 22:52

Result Grid Filter Rows: Search Export:

customer_id	total_purcha...
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

Query 13 : GROUP BY + MAX()

Finding the highest order placed by each customer

```
50 SELECT customer_id,
51      MAX(purch_amt) AS highest_purchase
52 FROM orders
53 GROUP BY customer_id;
```

100% 22:53

Result Grid Filter Rows: Search Export:

	customer_id	highest_purcha...
	3005	948.50
	3002	5760.00
	3009	2480.40
	3007	2400.60
	3001	270.65
	3004	1983.43
	3003	75.29
	3008	250.45

Query 14 : GROUP BY + HAVING

Company rewards salesmen whose total sales exceeds \$3,000

```
54 SELECT salesman_id,
55      SUM(purch_amt) AS total_sales
56 FROM orders
57 GROUP BY salesman_id
58 HAVING SUM(purch_amt) > 3000;
```

100% 30:58

Result Grid Filter Rows: Search Export:

	salesman_id	total_sales
	5001	11271.46

Query 15 : GROUP BY + HAVING

Find customers whose total purchases exceed \$2,500

```
59 SELECT customer_id,
60       SUM(purch_amt) AS total_purchase
61 FROM orders
62 GROUP BY customer_id
63 HAVING SUM(purch_amt) > 2500;
```

100% 30:63

Result Grid Filter Rows: Search Export:

customer_id	total_purcha...
3002	8870.86
3009	2590.90

Query 16 : GROUP BY + HAVING

Find customers who placed more than one order

```
60 SELECT customer_id,
61       COUNT(*) AS total_orders
62 FROM orders
63 GROUP BY customer_id
64 HAVING COUNT(*) > 1;
```

100% 21:64

Result Grid Filter Rows: Search

customer_id	tota...
3005	2
3002	3
3009	2

Query 17 : GROUP BY + HAVING + ORDER BY

Management wants top customers ranked by spending

```
62 SELECT customer_id,
63        SUM(purch_amt) AS total_purchase
64 FROM orders
65 GROUP BY customer_id
66 HAVING SUM(purch_amt) > 1000
67 ORDER BY total_purchase DESC;
68
```

100% 30:67

Result Grid Filter Rows: Search Export:

customer_id	total_purcha...
3002	8870.86
3009	2590.90
3007	2400.60
3004	1983.43
3005	1099.00

Query 18 : BETWEEN + HAVING

Find customers whose maximum purchase is between \$2,000 and \$6,000.

```
69 SELECT customer_id,
70        MAX(purch_amt) AS max_purchase
71 FROM orders
72 GROUP BY customer_id
73 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
74
75
```

100% 45:73

Result Grid Filter Rows: Search Export:

customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

Query 19 : COUNT + HAVING

Find salesmen who handled at least two orders

```
71 SELECT salesman_id,  
72        COUNT(*) AS total_orders  
73 FROM orders  
74 GROUP BY salesman_id  
75 HAVING COUNT(*) >= 2;  
76  
77
```

100% 1:76

Result Grid Filter Rows: Search Export:

salesman_id	tota...
5002	3
5001	4
5003	2

Query 20 : MAX + GROUP BY + HAVING

Management wants to daily highest purchases above \$2000

```
77 SELECT ord_date,  
78        MAX(purch_amt) AS highest_purchase  
79 FROM orders  
80 GROUP BY ord_date  
81 HAVING MAX(purch_amt) > 2000;  
82  
83
```

100% 30:81

Result Grid Filter Rows: Search Export:

ord_date	highest_purcha...
2012-10-10	2480.40
2012-07-27	2400.60
2012-09-10	5760.00
2012-04-25	3045.60